Homework 1: Data Structures in Python

Due Friday, 09.04.2026 at 11:59 PM ET

Goals

This homework has several objectives:

1. Write some basic Python programs.
2. Get familiar with the different data structures available in Python.
3. Leverage the concept of functions to write modular code.

Grading

This homework is worth 100 points:

1. **Problem 1 (20 points)**: `roster_overlap()` function
2. **Problem 2 (40 points)**: `histogram()` function
3. **Problem 3 (40 points)**: `moving_window_stats()` function

Instructions

In this homework, you need to write three Python functions, one per problem described below. All three function definitions are provided to you in `hw1.py`. DO NOT modify the function names or their parameters. You can use `given_tests.py` to test your functions in `hw1.py`. We have provided you with some test cases, you may make more of your own test cases and execute them to make sure your code runs properly.

Solve all problems using only Python's built-in types and functions. You may not import `numpy`, `pandas`, `scipy`, `matplotlib`, `statistics`, `sklearn`, `torch`, or any other external libraries. The autograder environment rejects a submission that imports any of these and gives it a 0, so check your file before you submit.

Problem 1

Create a function called `roster_overlap` that takes the rosters of several course sections and reports which students appear across them. This problem is designed to practice using sets together with dictionaries.

Your function should:

1. Have input argument `roster_overlap(sections)`, where `sections` is a dictionary whose keys are section names (strings) and whose values are lists of student names (strings). A student may appear in more than one section, and a name may be repeated within a single list.
2. Return a dictionary with exactly these three keys:
   - `'all_students'`: the student names that appear in at least one section.
   - `'in_every_section'`: the student names that appear in every section.
   - `'in_multiple_sections'`: the student names that appear in more than one section.
3. All three values must be sets, not lists. If `sections` is empty, all three sets should be empty.

For example, typing in

```
sections = {
    "Section1": ["ana", "ben", "cara"],
    "Section2": ["ben", "cara", "dan"],
    "Section3": ["cara", "dan"]
}
print(roster_overlap(sections))
```

should return

```
{
    'all_students': {'ana', 'ben', 'cara', 'dan'},
    'in_every_section': {'cara'},
    'in_multiple_sections': {'ben', 'cara', 'dan'}
}
```

Another test case is:

```
sections = {
    "Section1": ["eve", "eve", "frank"],
    "Section2": ["gina"]
}
print(roster_overlap(sections))
```

which returns

```
{
    'all_students': {'eve', 'frank', 'gina'},
    'in_every_section': set(),
    'in_multiple_sections': set()
}
```

Note that the order of the elements in a set does not matter, so your sets may print in a different order than shown above.

Problem 2

Create a function called `histogram` that takes as input a dictionary which contains the following fields: dataset `data`, a lower bound `min_val`, an upper bound `max_val`, and a number of bins `n`, and returns a histogram representation of `data` with `n` bins between these bounds. Specifically, your function should:

1. Have input arguments `histogram(input_dictionary)`, which, within the dictionary, it is expecting `data` as a list of floats, `n` as an integer, and `min_val` and `max_val` as floats.
2. Print the error message `'Error: min_val and max_val are the same value'` and return an empty list if `min_val` and `max_val` are the same number (the width of the histogram is 0).
3. Check that `n` is a positive integer. If it is not, your code should just return an empty list for `hist`.
4. If `min_val` is larger than `max_val`, re-assign `min_val` to `max_val` and `max_val` to `min_val`.
5. Initialize the histogram `hist` as a list of `n` zeros.
6. Calculate the bin width as `w = (max_val-min_val)/n`, so that `hist[0]` will represent values in the range `[min_val, min_val + w)`, `hist[1]` in the range `[min_val + w, min_val + 2w)`, and so on through `hist[n-1]`. (Remember that [is inclusive while) is not!)
7. Ignore any values in `data` that are less than or equal to `min_val`, or greater than or equal to `max_val`. Remember that if you swapped `max_val` and `min_val` in step 4, you need to work with the new values.
8. Increment `hist[i]` by 1 for each value in `data` that belongs to bin `i`, i.e., in the range `[min_val + i*w, min_val + (i+1)*w)`.
9. Return `hist`.

For example, typing in

```
data = [-2, -2.2, 0, 5.6, 8.3, 10.1, 30, 4.4, 1.9, -3.3, 9, 8]
input_dictionary = {'data': data, 'n': 15, 'min_val': -5, 'max_val': 10}
hist = histogram(input_dictionary)
print(hist)
```

should return

```
[0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 2, 1]
```

Some other test cases are:

```
data = [-4, -3.2, 0, 7.6, 1.0, 2.2, 30, 2.2, 1.9, -8.3, 6, 5]
input_dictionary = {'data': data, 'n': 10, 'min_val': 10, 'max_val': 0}
hist = histogram(input_dictionary)
print(hist)
```

should return

```
[0, 2, 2, 0, 0, 1, 1, 1, 0, 0]
```

and,

```
data = [2,2,2]
input_dictionary = {'data': data, 'n': 5, 'min_val': -2, 'max_val': 3}
hist = histogram(input_dictionary)
print(hist)
```

returns

```
[0, 0, 0, 0, 3]
```

also,

```
data = [-1,-1,-1,10,10]
input_dictionary = {'data': data, 'n': 5, 'min_val': -1, 'max_val': 10}
hist = histogram(input_dictionary)
print(hist)
```

returns

```
[0, 0, 0, 0, 0]
```

Note: Please include all conditions specified in this problem into your code.

Problem 3

Create a function called `moving_window_stats` that processes time-series data from various sensors to calculate moving statistics. This problem is designed to practice iterating through lists and using list slicing to analyze subsets of data.

Your function should:

1. Have input arguments `moving_window_stats(data_dict, k)`, where:
   - `data_dict` is a dictionary where the keys are sensor names (strings) and the values are lists of numerical readings (floats or integers).
   - `k` is a positive integer representing the window size. You may assume `k >= 1`.
2. For each sensor in `data_dict`:
   - Check if the length of its data list is sufficient (at least `k`). If the list is shorter than `k`, the result for that sensor should be an empty list `[]`.
   - Iterate through the data list to create "windows" of size `k`. It's recommended that you use list slicing (e.g., `current_window = data[start:end]`) to extract the values for each window.
   - For each window, calculate three statistics:
     - The minimum value in the window.
     - The maximum value in the window.
     - The average (mean) value of the window (sum divided by count).
   - Store these statistics as a tuple, in this order: `(min_val, max_val, average_val)`.
3. Return a new dictionary where the keys are the original sensor names and the values are lists of the calculated tuples.

The no-external-libraries rule at the top of this handout applies here: use Python's built-in functions on each window.

For example, if the input is:

```
data = {
    "Sensor_A": [1, 2, 3, 4, 5],
    "Sensor_B": [10, 20]
}
k = 3
```

The logic proceeds as follows:

- **Sensor_A**:
  - Window 1 (Indices 0-2): `[1, 2, 3]` -> Min: 1, Max: 3, Avg: 2.0. Tuple: `(1, 3, 2.0)`
  - Window 2 (Indices 1-3): `[2, 3, 4]` -> Min: 2, Max: 4, Avg: 3.0. Tuple: `(2, 4, 3.0)`
  - Window 3 (Indices 2-4): `[3, 4, 5]` -> Min: 3, Max: 5, Avg: 4.0. Tuple: `(3, 5, 4.0)`
- **Sensor_B**: The list length (2) is less than `k` (3), so the result is `[]`.

The function should return:

```
{
    "Sensor_A": [(1, 3, 2.0), (2, 4, 3.0), (3, 5, 4.0)],
    "Sensor_B": []
}
```

Another test case is:

```
data = {"Temp": [20.0, 21.0, 22.5]}
k = 2
print(moving_window_stats(data, k))
```

which returns

```
{'Temp': [(20.0, 21.0, 20.5), (21.0, 22.5, 21.75)]}
```

and, with a window size of 1, every reading forms its own window:

```
data = {"Pressure": [3, 7, 4]}
k = 1
print(moving_window_stats(data, k))
```

returns

```
{'Pressure': [(3, 3, 3.0), (7, 7, 7.0), (4, 4, 4.0)]}
```

Testing

We have provided test cases for you in `given_tests.py`, which contains tests for all three problems as outlined in this readme. Note that these test programs will only work "out of the box" if you have your solution in `hw1.py`. You may verify your code by running the test program from the terminal:

```
python given_tests.py
```

The concept of importing functions from modules or `.py` files is being used here. You can also modify the test cases or add your own to further test your implementation.

These are not the tests we grade with. The autograder runs additional cases, so passing everything in `given_tests.py` does not guarantee full credit. Work through the conditions listed above and test the edge cases yourself.

What to Submit

Please submit ONLY `hw1.py` with all the appropriate functions filled in for Problems 1, 2 and 3.

Before submitting, make sure to test your code by running:

```
python given_tests.py
```

This will help you verify that your functions work correctly and that your file imports properly.